

Code Review : MinD-Vis Training Code for fMRI Vision Decoding, Focusing on SC-MBM and DC-LDM

날짜 : 2025년 12월 21일

과목 : 딥러닝의 기초와 프로그래밍

교수 : 강동훈 교수님

소속 : UST KIST School, 바이오 메디컬 융합 전공

학번 : 02523057

이름 : Ryu Hanyeong

선정 논문

Title : Seeing Beyond the Brain: Conditional Diffusion Model with Sparse Masked Modeling for Vision Decoding

Author : Zijiao Chen, Jiaxin Qing, Tiange Xiang, Wan Lin Yue, Juan Helen Zhou

Conference: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023

Link: <https://mind-vis.github.io>

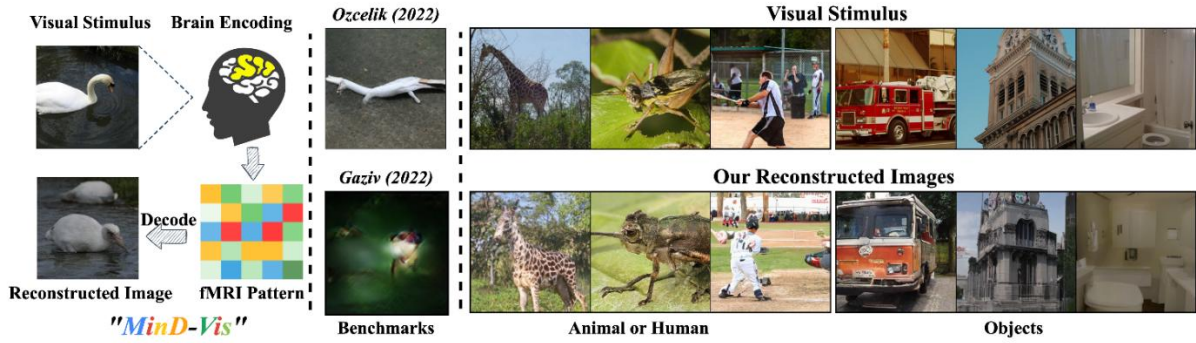


Figure 1 Brain Decoding and Image Reconstruction

1. Summary

1.1. Background and Problem

Decoding visual experiences from fMRI is major goal of Brain Computer Interfaces (BIC). However existing deep learning-based approaches suffer from data acarcity, individual differences, and information mismatch. Medical data collection is chronically difficult. Firstm the scale of paired fMRI-image datasets is too small to train effective models using conventional methods. Second, individual variability in brain activation patterns like location and size of activated regions for the same visual stimulus makes it difficult to build a generalized model. , making it hard to train models using conventional methods. Finally, the difficulty of restoring images with a large amount of information from fMRI voxel data with a small amount of information is high.

1.2 Proposed Method

To solve these problems, this paper proposes the MinD-Vis framework, which combines self-supervised learning of brain signal context with a Latent Diffusion Model (LDM). This model consists of two sequential training stages.

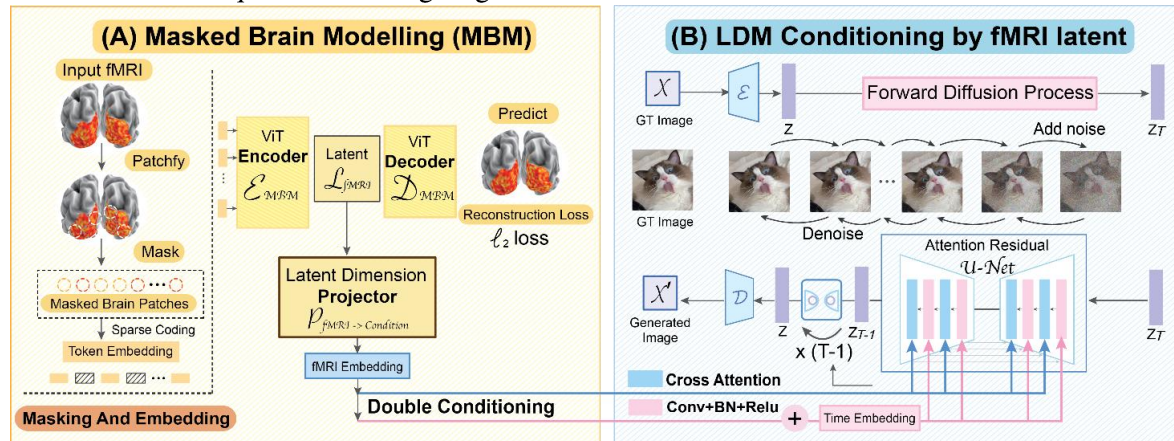


Figure 2 The Two-Stage Architecture of MinD-Vis : SC-MBM and DC-LDM

Stage A : Sparse-Coded Masked Brain Modeling (SC-MBM)

This stage strains the model using only fMRI data without images. They divide fMRI signals into patches and masked 75% of them. Then, an autoencoder trained to recover these masked part. The model uses a large embedding size (1024) compared to the input patch. This mimics the brain's "Sparse Coding" principle, allowing the model to store more information efficiently.

Stage B: Double-Conditioned Latent Diffusion Model (DC-LDM)

This stage generates images using the pre-trained fMRI model. We use a pre-trained LDM (based on U-Net) and apply "Double Conditioning" to make the result accurate. First is Cross-Attention. It uses fMRI data to guide the detailed content of the image. Second is Time-Step Conditioning. It adds fMRI features to the time steps to maintain the global context of the brain signal.

1.3. Significance

As a result, MinD-Vis achieved high Semantic Accuracy and generation quality (FID) even with a small amount of paired data. This study is evaluated as a successful case that solves the data scarcity problem while effectively reflecting biological principles (Sparse Coding) into the design of a deep learning model.

2. Analysis of Training Code

2.1 Stage A : Pre-training with Masked Modeling (SC-MBM)

`code/sc_mbm/mae_for_fmri.py` has the process for pre-training with masked modeling. Due to space constraints, I focus on the self-supervised learning process and the code.

2.1.1 1D patch Embedding (Sparse Coding implementation)

```
class PatchEmbed1D(nn.Module):
    def __init__(self, num_voxels=224, patch_size=16, in_chans=1,
embed_dim=768):
        ...
        self.proj = nn.Conv1d(in_chans, embed_dim, kernel_size=patch_size,
stride=patch_size)
    def forward(self, x, **kwargs):
        B, C, V = x.shape # batch, channel, voxels
        # assert V == self.num_voxels, \
        #     f"Input fmri length ({V}) doesn't match model
({self.num_voxels})."
        x = self.proj(x).transpose(1, 2).contiguous() # put embed_dim at
the last dimension
        return x
```

It divides 1D fMRI voxels into patches using a convolutional layer.

```
class MAEforFMRI(nn.Module):
    def __init__(self, num_voxels=224, patch_size=16, embed_dim=1024,
in_chans=1,
        ...
        self.patch_embed = PatchEmbed1D(num_voxels, patch_size, in_chans,
embed_dim)
```

In the MAEforFMRI class, `embed_dim` is set to 1024, whereas the input `patch_size` is typically small. This large embedding-to-patch ratio implements the Sparse Coding principle mentioned in the paper (Section 3.2), projecting fMRI data into a high-dimensional space to increase information capacity.

2.1.2 Random Masking

```
def random_masking(self, x, mask_ratio):
    N, L, D = x.shape # batch, length, dim
    len_keep = int(L * (1 - mask_ratio))
```

The `random_masking` function in the MAEforFMRI class implements the core self-supervised

learning objective. It generates random noise to shuffle patches and keeps only a subset based on the mask_ratio.

```
noise = torch.rand(N, L, device=x.device) # noise in [0, 1]
if self.focus_range is not None:
    for i in range(N):
        noise[i, ids_mask[i,:]] = 1.1 # set mask portion to 1.1
```

For each patch, generate a random value in the range [0,1]. If there is a focus_range, a specific range is designated as the mask target and the mask portion is set to 1.1.

```
# sort noise for each sample
ids_shuffle = torch.argsort(noise, dim=1) # ascend: small is
keep, large is remove
ids_restore = torch.argsort(ids_shuffle, dim=1)

# keep the first subset
ids_keep = ids_shuffle[:, :len_keep]
x_masked = torch.gather(x, dim=1, index=ids_keep.unsqueeze(-
1).repeat(1, 1, D))

# generate the binary mask: 0 is keep, 1 is remove
mask = torch.ones([N, L], device=x.device)
mask[:, :len_keep] = 0
# unshuffle to get the binary mask
mask = torch.gather(mask, dim=1, index=ids_restore)

return x_masked, mask, ids_restore
```

ids_shuffle creates a mask by selecting len_keep number of original indices sorted in ascending order by noise value. Ids_shuffle is a mask to return the shuffled order to the original order. This code confirms that the model randomly removes a large portion of the data (set to 75% via mask_ratio in forward part) and forces the encoder to reconstruct it, leveraging the spatial redundancy of fMRI signals.

2.1.3 Asymmetric Encoder-Decoder Architecture

The MAEforFMRI class adopts an asymmetric design as described in the paper.

```
def forward_encoder(self, x, mask_ratio):
    x = self.patch_embed(x)
    x = x + self.pos_embed[:, 1:, :]

    x, mask, ids_restore = self.random_masking(x, mask_ratio)

    cls_token = self.cls_token + self.pos_embed[:, :1, :]
    cls_tokens = cls_token.expand(x.shape[0], -1, -1)
    x = torch.cat((cls_tokens, x), dim=1)

    for blk in self.blocks:
        x = blk(x)
    x = self.norm(x)

    return x, mask, ids_restore
```

This function segment and embed patches using Conv1d, add positional embeddings, perform random masking, add positional embeddings to CLS tokens, and greet the surroundings. Then it

encode using Transformer and apply Layer Normalization.

```
def forward_decoder(self, x, ids_restore):
    x = self.decoder_embed(x)
    mask_tokens = self.mask_token.repeat(x.shape[0],
ids_restore.shape[1] + 1 - x.shape[1], 1)
    x_ = torch.cat([x[:, 1:, :], mask_tokens], dim=1) # no cls token
    x_ = torch.gather(x_, dim=1, index=ids_restore.unsqueeze(-
1).repeat(1, 1, x.shape[2])) # unshuffle
    x = torch.cat([x[:, :1, :], x_], dim=1) # append cls token
    x = x + self.decoder_pos_embed
    for blk in self.decoder_blocks:
        x = blk(x)
    x = self.decoder_norm(x)
    x = self.decoder_pred(x)
    x = x[:, 1:, :]
    return x
```

Conversely, the decoder takes the encoded latent vectors and appends learnable mask_tokens to reconstruct the original signal. This structure aligns with the standard MAE approach tailored for fMRI data. The encoder utilizes deeper layers and higher-dimensional embedding dimensions, as this process focuses on representation learning. Actual image generation occurs in Stage B.

2.2. Stage B: Double-Conditioned LDM (DC-LDM)

2.2.1 Encoding fMRI Conditions

In this stage, model makes the image from fMRI signal with double conditioning.

```
class cond_stage_model(nn.Module):
    def __init__(self, metafile, num_voxels, cond_dim=1280,
global_pool=True):
        ...
        model = create_model_from_config(metafile['config'], num_voxels,
global_pool)
        ...
        self.dim_mapper = nn.Linear(self.fmri_latent_dim, cond_dim,
bias=True)
        ...
```

lcond_stage_model class in code/dc_ldm/ldm_for_fmri.py uses the pre-trained MAE encoder from Stage A to extract latent representations. It encode fMRI data with pre-trained MAE and map the dimension to cond_dim.

2.2.2 Double Conditioning Mechanism

```
class UNetModel(nn.Module):
    def forward(self, x, timesteps=None, context=None, y=None, **kwargs):
        if self.use_time_cond: # add time conditioning
            c = self.time_embed_condtion(context)
            assert c.shape[1] == 1, f'found {c.shape}'
            emb = emb + torch.squeeze(c, dim=1)
        ...
```

In code/dc_ldm/modules/diffusionmodules/openaimodel.py, the encoded fMRI features (context) are transformed and added to the time embedding(emb). This enforces a consistent global semantic guidance across all denoising steps.

```

class CrossAttention(nn.Module): # Optimize this module as well
    def __init__(self, query_dim, context_dim=None, heads=8, dim_head=64,
dropout=0., cond_scale=1.):
        self.to_q = nn.Linear(query_dim, inner_dim, bias=False)
        self.to_k = nn.Linear(context_dim, inner_dim, bias=False)
        self.to_v = nn.Linear(context_dim, inner_dim, bias=False)
...
    def forward(self, x, context=None, mask=None):
        q, k, v = map(lambda t: rearrange(t, 'b n (h d) -> (b h) n d',
h=h), (q, k, v))
        sim = einsum('b i d, b j d -> b i j', q, k) * self.scale
...

```

In code/dc_ldm/modules/attention.py, the context (fMRI data) is used to generate Key (K) and Value (V) matrices, while the image latent features serve as Query (Q). This allows the model to attend to specific brain signal patterns while generating image details.

3. Conclusion

In this report, I verified how the core components of the paper, SC-MBM and DC-LDM, are actually implemented in the training code. Analyzing the source code allowed me to see how the theoretical processes described in the paper are translated into practical algorithms, bridging the gap between theory and real-world research application.

Personally, I have also been struggling with the scarcity of brain signal and medical data, as well as the challenge of extracting precise features from such complex signals. The masked modeling pre-training strategy used in this paper appears to be a promising solution to these problems. I was particularly interested by how biological features were mapped to embedding dimensions which I learned in class.

In the future, AI that expresses brain signals using this method will be able to provide more objective interpretations of stimuli.